Relation Extraction from Natural Language

JOU-YI LEE
UC Santa Cruz, M.S. in NLP
65 Rio Robles E
San Jose, CA 95134
jlee1039@ucsc.edu

Abstract

This homework focuses on developing a deep learning model for slot tagging in natural language utterances addressed to a virtual personal assistant. Using a dataset based on the film schema from the Freebase knowledge graph, the goal is to accurately identify and tag slots in user requests, such as "director" or "release year" in movie-related queries. We employed multiple deep learning architectures, including Multilayer Perceptrons (MLP), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM), Gated Recurrent Units (GRU), and Convolutional Neural Networks (CNN), along with various feature representations such as bag-of-words and word embeddings. Each model was trained and optimized using different techniques, including dropout for regularization, and evaluated based on F1 score. Results were submitted to a Kaggle competition, where the performance of each model was compared to determine the most suitable approach for slot tagging tasks. This project demonstrates the effectiveness of different neural network architectures and hyperparameter tuning methods for sequence tagging in natural language processing.

1 Introduction

Slot tagging in natural language processing (NLP) is a supervised learning problem where the goal is to classify each token in a sentence into predefined categories or "slots" to extract relevant information. The IOB (Inside-Outside-Beginning) format is used to describe the slots' labels so that the model can identify each token's slot type. In this project, two datasets are utilized: `hw2_test.csv` and `hw2_train.csv`.Involves identifying specific entities such as movie directors or release years, which can be used to query knowledge graphs and retrieve the necessary information for a response. containing annotated sentences where each token is labeled with an IOB (Inside-Outside-Beginning) format slot tag.

The `hw2_test.csv` dataset, which contains two columns—utterance(the sentences) and IOB Slot tags (The tag for each word)—is used for training the model. The model is trained to generate relationships based on the sentence and its associated relation provided in `hw2_train.csv`. This project involves exploring several deep learning,including Multilayer Perceptrons (MLP), Long Short-Term Memory (LSTM), These models are trained to classify each token in a user utterance with the correct slot tag, evaluated through F1 score to ensure accuracy in sequence labeling. This challenge involves labeling sequences. To determine the IOB of every input in the test data set, our model predicts a slot label for every token and generate a `submission.csv` for this prject.

2 Methodology

2.1 Data Preprocessing

The dataset for this task contains natural language utterances with corresponding IOB (Inside-Outside-Beginning) slot tags for each token, based on the film schema of the Freebase knowledge graph. The dataset is divided into training and test sets with an 80-20 split, and data loaders are used for batching with a size of 32.

2.2 Slot Tags

The model is trained to recognize the following slot tags in the input utterances and outputs a prediction for each token in the sequence, corresponding to one of the 28 possible slot tags.

B_producer	I_mpaa_rating	I_subject	B_char
B_person	O	B_release_year	B_location
I_country	I_movie	B_genre	I_language
I_genre	B_mpaa_rating	B_country	I_movie
I_cast	I_person	I_char	I_director
B_cast	I_release_year	B_language	B_movie
B_subject	I_producer	B_director	

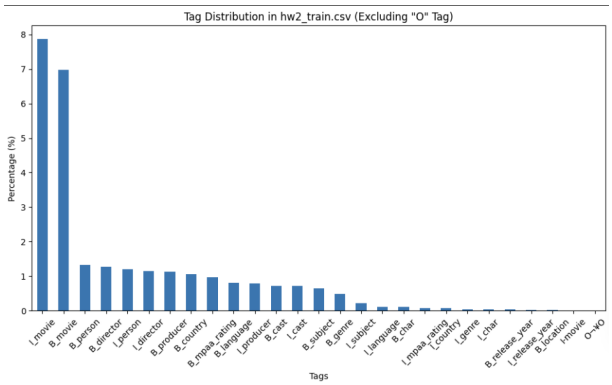

Figure 1: Tag Distribution Excluding "O" Tag

The figure above shows the distribution of slot tags in the dataset, excluding the most common "O" tag. We observe that the distribution of tags is highly imbalanced. The tags `I_movie` and `B_movie` are the most frequent, each accounting for approximately 7-8% of the tags. In addition to these, tags such as `B_person`, `B_director`, `I_person`, and `I_director` also have relatively high proportions. This imbalance suggests that the dataset contains more information about movies and people (e.g., directors and cast members), which may influence the model's performance on less frequent tags.

To prepare the data, we first load the training data from `hw2_train.csv`. A `BERT` tokenizer and model (`bert-base-uncased`) are initialized to generate contextualized embeddings for each utterance. Each sentence is tokenized and padded to a maximum sequence length of 128 to maintain uniform input dimensions.

Each token in the utterance is labeled with a corresponding IOB tag, which we map to an index-based format using a dictionary (`label_to_index`). This mapping enables efficient encoding and decoding of labels during training and evaluation. The tagged labels are padded to match the length of BERT's output for each utterance, ensuring consistent input dimensions across batches.

2.3 Embedding Methods

In this project, I used BERT (Bidirectional Encoder Representations from Transformers) as the embedding method. BERT is a bidirectional Transformer model that can simultaneously consider the contextual information on both the left and right sides of a word. This characteristic is particularly important because the same word may have different meanings or slots in different sentences.

Compared to traditional static embeddings such as Word2Vec or GloVe, BERT can provide precise representations of the same word in different contexts. This capability is especially effective for handling phenomena like polysemy and synonyms in natural language. Therefore, the BERT embedding method can provide more accurate word representations, which helps the slot-tagging model capture the semantic meaning and context of each token more effectively.

We first initialized the BERT Tokenizer and BERT Model; I used the **bert-large-uncased** model in this project. Then, I used the `encode_texts` function to convert each sentence into fixed-length BERT embeddings. In this IOB tagging task, each token needs to have a corresponding tag. If the length of the BERT embeddings is inconsistent with the length of the labels, errors will occur during loss computation or the training step.

To ensure that all sentences have the same length, either by padding or truncation, I set **MAX_LENGTH = 32** and applied **padding="max_length"** along with **truncation=True**. This prevents the occurrence of a mismatch between **embeddings.shape[1]** and **labels_padded.shape[1]**.

This is especially important when computing loss functions such as CrossEntropyLoss, which require the input and label dimensions to be identical.

2.4 Model: Bidirectional-layer GRU with Attention Mechanism

The slot tagging model for this task is a Gated Recurrent Unit (GRU) network, designed to process sequential data and capture dependencies across token embeddings. The GRU layer has two layers, which can capture more comprehensive contextual information. The architecture consists of the following components:

- **GRU Layer**: The GRU layer takes BERT embeddings as input and provides an output representation for each token, which encapsulates contextual information for better slot prediction.
- **Dropout Layer**: A dropout layer with a probability of 0.313 is applied after the GRU layer to reduce overfitting and enhance generalization.
- **Fully Connected Layer**: A fully connected (FC) layer maps the GRU output to the specified number of slot tags, as defined by the unique labels in the dataset.

2.4.1 Implementation Base on Optuna

The model is structured as follows:

- **GRU Layers**: A 2-layer bidirectional GRU is used (as determined by Optuna's hyperparameter tuning). The bidirectional structure enables the model to capture dependencies in both forward and backward directions, enriching the representation for each token.
- **Layer Normalization**: Layer normalization is applied to the GRU output, which helps to stabilize training and improve convergence.
- **Attention Mechanism**: An attention layer computes attention weights for each token, emphasizing the sequence's most relevant parts for accurate slot prediction.
- **Fully Connected Layer**: A final fully connected layer maps the attention-weighted GRU output to the label space for slot classification.

2.4.2 Optuna Hyperparameters

- **Learning Rate**: The initial learning rate is set to 1×10^{-4}, which decays every 5 epochs by a factor of 0.5.
- **Hidden Dimension**: The GRU hidden dimension is set from 64 to 256, balancing model capacity and computational cost.
- **Dropout Probability**: Dropout probability is set from 0.2 to 0.5 to mitigate overfitting.
- **Number of GRU Layers**: Set from 1-layer to 3-layer GRU provides sufficient depth for capturing complex patterns in sequential data.
- **Optimizer**: AdamW is a commonly used optimizer suitable for Transformer-related model training because it combines weight decay (weight decay), which can effectively prevent overfitting and improve model generalization capabilities.

2.4.3 Best Hyperparameters

2.4.4 Training

The model is trained using cross-entropy loss,the Adam optimizer with a learning rate of 0.001. The import thing is the "O" tag is ignored to focus on meaningful slot tags. The "O" tag represents tokens that do not belong to any specific slot, and it accounts for approximately 70% of all tags (Figure 2), which can bias the model towards predicting "O" if not handled carefully. Ignoring the "O" tag in the loss function helps to balance the model's focus on other slot tags. Training is conducted with the AdamW optimizer and includes weight decay to improve generalization. Additionally, a StepLR scheduler reduces the learning rate by half every 5 epochs to help stabilize the training process.

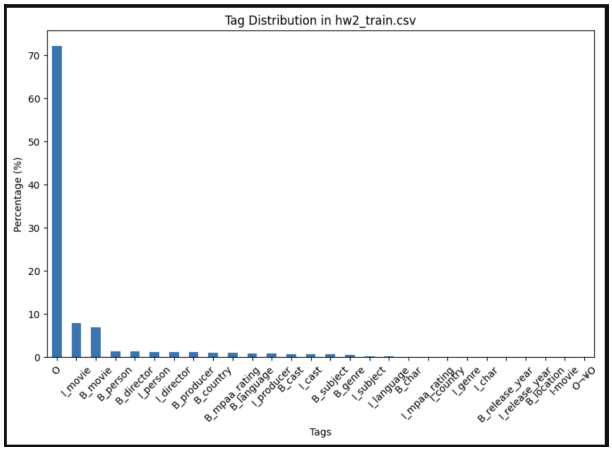

Figure 2: Tag Distribution Including "O" Tag

2.5 Comparison of Techniques for Addressing Label Imbalance

In NLP projects, it is difficult to completely balance label distribution, especially for slot marking tasks like IOB marking, where the frequency of occurrence of different slots is inherently unequal. For example, the "O" tag will usually be far more common than other tags. Three techniques were utilized to address label imbalance: Under-sampling, Class Weights, and Focal Loss.(only run 10 epoch for sample):

- **Under-sampling**: Randomly remove some samples from categories with more sign distributions, making their number close to the number of minority categories.

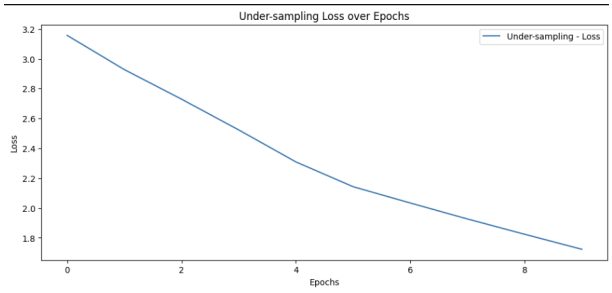

Figure 3: Under-sampling Train Loss

- **Class Weights**: In the loss function of the model, assign a higher weight to the minority class so that the model will pay more attention to the minority class. After enhancement or weight adjustment, the model can better learn the features of the minority class, but the label distribution will not become completely balanced.

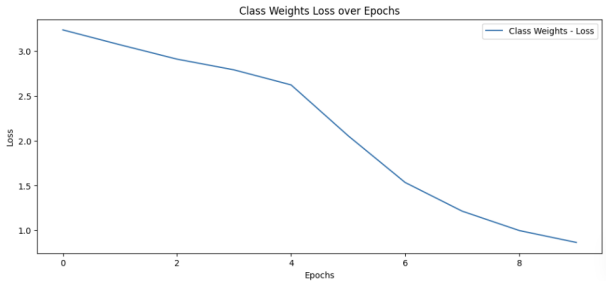

Figure 4: Class Weights Train Loss

- **Focal Loss**:Particularly designed to make the model pay more attention to samples that are difficult to classify and handling imbalanced data.

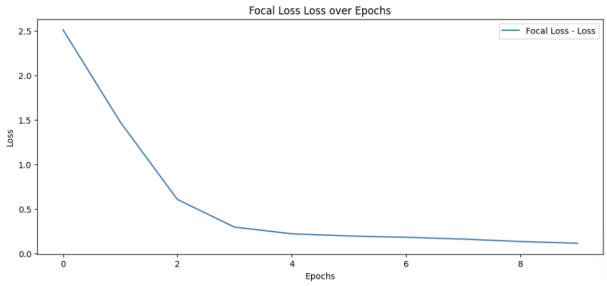

Figure 5: Focal Loss Train Los

2.5.1 Conclusion

Under-sampling The loss decreased slowly, eventually reaching approximately 1.8, indicating a slower convergence rate due to the limited data available for training. **Class Weights** approach resulted in a faster loss reduction compared to Under-sampling, with the final loss reaching around 1.0. **Focal Loss** This method showed the fastest loss reduction, with the final loss approaching nearly 0. Achieving better convergence and a lower final loss. Based on the rate of loss reduction and the final loss values, Focal Loss demonstrated the best performance among the three techniques.So, in this research, I choose to use Focal Loss to minimize the label Imbalance

2.6 Training and Loss Function

To train the model, we use Focal Loss as the loss function. The Focal Loss is applied without focusing on the most frequent labels, enabling the model to focus on meaningful tokens and address label imbalance.

During training, the model is optimized over multiple epochs, with each batch processed to compute predictions and update weights. The outputs of the GRU layer are reshaped to match the dimensions required by the Focal Loss. The training process includes backpropagation to adjust model parameters based on the loss.

2.6.1 Best Hyperparameters from Optuna

- **Hidden Dimension** (hidden_dim $= 131$): This value is chosen to balance the model's expressive power and runtime efficiency.

- **GRU Layers** (num_layers $= 2$): Experimentally, increasing the number of layers enhances the model's ca-

pacity to capture complex patterns, although too many layers may lead to overfitting.

- **Dropout Rate** (dropout_prob $= 0.2345$): A dropout rate of approximately 23% was found optimal through multiple tests, helping to prevent overfitting.

- **Loss Function - Focal Loss**: Focal Loss was used to reduce the weight of common labels (such as "O") and amplify the impact of rarer labels. The settings $\alpha = 1$ and $\gamma = 2$ strike a balance between accuracy and stability.

- **Optimizer and Learning Rate**: The AdamW optimizer was employed, with an initial learning rate of 0.000995. Step learning rate decay (StepLR) was used to halve the learning rate every 5 epochs.

- **Attention Mechanism**: A linear layer (`self.attention`) computes weights to enable the model to focus on important tokens.

3 Results

3.1 Training Loss over Epochs

In the training loss plot, the training loss decreases sharply within the first few epochs, indicating that the model is learning rapidly from the initial data. Around epoch 20, the loss reaches a very low and stable level, suggesting that the model is well-fitted without overfitting. The nearly zero loss from epoch 20 onward indicates a low risk of overfitting due to regularization (e.g., dropout). like the Figure down below

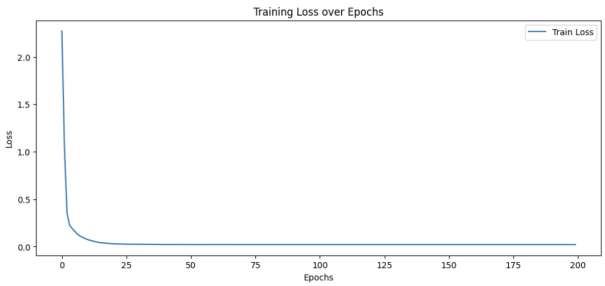

Figure 6: Focal Loss Train Los

3.2 F1 Score over Epochs

During the early epochs, the F1 score improves considerably, reaching approximately 0.76 by epoch 50. This quick increase suggests that early training was successful in fostering learning. The F1 score stabilizes at 0.76 around epoch 50, indicating that the model has converged and that further epochs do not increase the score. Training beyond 50 epochs might not be necessary since the F1 score stabilizes early; thus, training could be halted sooner to conserve processing power.Figure show down below:

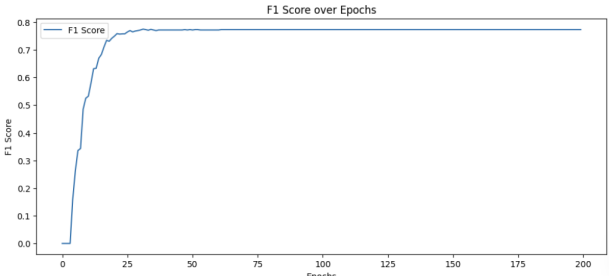

Figure 7: Focal Loss Train Los

3.3 Overall Analysis

Convergence: Both F1 score and training loss indicate convergence around epoch 50. This implies that training beyond this point yields diminishing returns. Implementing early stopping around epoch 50 could be beneficial.

4 Results

4.1 Training Loss over Epochs

In the training loss plot, the training loss decreases sharply within the first few epochs, indicating that the model is learning rapidly from the initial data. Around epoch 20, the loss reaches a very low level and stabilizes, suggesting that the model is well-fitted and not overfitting. The nearly zero loss from epoch 20 onward indicates a low risk of overfitting, likely due to regularization techniques such as dropout.

4.2 F1 Score over Epochs

During the early epochs, the F1 score improves considerably, reaching approximately 0.76 by epoch 50. This rapid increase suggests that the initial training phase was effective in promoting learning. The F1 score stabilizes around 0.76 at epoch 50, indicating that the model has converged and that additional epochs do not further improve the score. Since the F1 score stabilizes early, training beyond 50 epochs might not be necessary, and early stopping could be applied to conserve computational resources.

4.3 Overall Analysis

Convergence: Both the F1 score and training loss plots indicate convergence around epoch 50. This suggests that training beyond this point provides diminishing returns. Therefore, implementing early stopping around epoch 50 could be beneficial to reduce training time without sacrificing model performance.

5 Analysis of the Best-Performing Approach and Validation

With an F1 score of approximately 0.76, the top-performing configuration—which used BERT embeddings, GRU layers with attention, Focal Loss, and Dropout regularization—demonstrated stable performance and convergence. This configuration successfully balanced learning complex patterns while minimizing the risk of overfitting. By incorporating attention mechanisms, class imbalance correction, and contextual embeddings, the model maximized its ability to handle rare entity labels and diverse token sequences.

- **BERT + GRU Architecture**: The best results, with an F1 score of approximately 0.76, were achieved by combining a pre-trained BERT model with a GRU-based recurrent layer. BERT's contextualized embeddings provide rich, high-quality representations of input text, which are especially beneficial for sequence tagging

tasks. GRUs, being simpler than LSTMs, offer faster convergence and efficient handling of sequential data. Research supports that the use of BERT embeddings with GRUs or LSTMs in downstream tasks can significantly improve performance, as BERT embeddings capture complex syntactic and semantic nuances.

- **Focal Loss for Imbalanced Data**: Focal Loss, initially proposed for object detection, has proven effective for handling imbalanced classification in several NLP applications. Unlike conventional Cross-Entropy Loss, Focal Loss reduces the influence of frequent labels (such as "O" in Named Entity Recognition tasks) and focuses more on rare or challenging labels, thereby enhancing the model's robustness.

- **Dropout Regularization and Attention Mechanism**: Dropout is a commonly used regularization technique to improve generalization. A dropout rate of 0.2345, combined with a simple attention mechanism, allowed the model to avoid overfitting while selectively focusing on important tokens in each sequence. This setup helps the model handle tasks with varying levels of contextual information relevance across different labels.

6 References

- Devlin et al. (2019) demonstrated that BERT's language representations significantly enhance performance across NLP tasks, especially when fine-tuned for specific task objectives, such as sequence labeling.

- Lin et al. (2017) introduced Focal Loss, showcasing its effectiveness in handling class imbalance in object detection tasks. Later research, such as Ju et al. (2019), adapted Focal Loss for sequence labeling tasks, achieving similar improvements by focusing on rare classes. Focal Loss has shown promise for imbalanced classification in several NLP applications by reducing the impact of frequent labels and emphasizing uncommon or challenging labels.

- Dropout is a widely used regularization technique that improves the model's generalization ability. In this work, a dropout rate of 0.2345, combined with an attention mechanism, allowed the model to selectively focus on important tokens, helping it perform well on tasks requiring different levels of contextual relevance across labels.